Step 3 - System Counter Architecture 1
Reference Note 1
Learning Objectives 1
1. What is the System Counter? 1
2. System Counter Consumers 2
3. Why ARM Uses a Shared System Counter 2
Benefits 2
4. Key Properties of the System Counter 2
5. Monotonic Behavior 3
6. Fixed Frequency Operation 3
7. Independent of CPU Frequency 4
Example 4
8. Always-On Domain 4
Why? 5
Benefit 5
9. System Counter in a Multi-Core System 5
Benefits 5
10. Live Counter Behavior 6
Explanation 6
11. Why the System Counter Matters 6
Chapter Summary 7
Knowledge Check 8
Q1 8
Q2 8
Q3 8
Q4 8
Q5 8
Q6 8
Answer Key 8

Step 3 - System Counter Architecture

Reference Note

This chapter is based on concepts described in the ARM Generic Timer Architecture and Cortex-A Generic Timer architecture documentation.

The System Counter is the foundation of the ARM Generic Timer Architecture. All counters and timers derive their notion of time from the System Counter.

Learning Objectives

After completing this chapter, you should understand:

- What the System Counter is
- Why ARM uses a shared System Counter
- The key properties of the System Counter
- Why it is independent of CPU frequency
- Why it resides in an Always-On domain
- How software observes the System Counter
- Introduction to Counter Scaling

1. What is the System Counter?

The System Counter is the master source of time within an ARM system.

Conceptually:

0 → 1 → 2 → 3 → 4 → 5 → ...

The count continuously increases and serves as the foundation for all timing operations.

2. System Counter Consumers

System Counter Consumer	Purpose
Physical Counter	Direct view of system time
Virtual Counter	Virtualized view of system time
Physical Timer	Generates events using physical time
Virtual Timer	Generates events using virtual time

3. Why ARM Uses a Shared System Counter

Instead of implementing:

Core0 Counter
Core1 Counter
Core2 Counter
Core3 Counter

ARM uses:

One System Counter
Shared by all cores

Benefits

Design Choice	Benefit
One System Counter	Common notion of time
Shared across cores	No clock synchronization needed
Common timestamp source	Consistent time measurements
Single time base	Simplified timeout calculations

4. Key Properties of the System Counter

Property	Description
Monotonic	Counter only increments
Shared	Same time source for all processor cores
Fixed Frequency	Runs at a constant rate
Always-On	Continues running during CPU idle states
Architecture Defined	Standardized by ARM Generic Timer architecture

5. Monotonic Behavior

The System Counter only moves forward.

Example:

100
101
102
103
104

The count must never move backwards.

Guarantee:

Current Time ≥ Previous Time

This property is fundamental for:

- Timestamping
- Timeout calculations
- Event scheduling

6. Fixed Frequency Operation

The System Counter operates at a fixed frequency.

Example:

24 MHz

This means:

24,000,000 counter increments per second

The frequency remains constant regardless of processor activity.

7. Independent of CPU Frequency

A common misconception:

CPU Clock = System Counter Clock

False.

Example

Scenario	CPU Frequency	System Counter Frequency
State 1	1 GHz	24 MHz
State 2	2 GHz	24 MHz

Result:

CPU speed changes
Timekeeping remains unchanged

This allows reliable timing even when processor frequency changes.

8. Always-On Domain

The System Counter typically resides in an Always-On power domain.

Why?

Time must continue advancing even when:

CPU Idle
CPU Sleep
Low-Power States

occur.

Benefit

CPU State	Counter Running?
Active	Yes
Idle	Yes
Sleep	Yes
Low-Power State	Yes

Without this property:

Timeout detection becomes unreliable.

9. System Counter in a Multi-Core System

Core	Observes
Core 0	Shared System Counter
Core 1	Shared System Counter
Core 2	Shared System Counter
Core 3	Shared System Counter

Benefits

Feature	Result
Task Migration	Consistent timestamps
Inter-Core Communication	Same timeline
Time Measurement	Comparable across all cores

10. Live Counter Behavior

The System Counter never stops.

Example:

MRS X0, CNTPCT_EL0
MRS X1, CNTPCT_EL0

Possible result:

Read	Value
First Read	1,000,000
Second Read	1,000,015

Explanation

Observation	Reason
Values differ	Counter advanced between reads

This behavior is expected.

11. Why the System Counter Matters

Everything in the Generic Timer Architecture ultimately references the System Counter.

Examples:

- Time measurement
- Timer programming
- Interrupt generation
- Virtualization
- Timeout calculations

Think of the System Counter as:

The root of the timer architecture.

Chapter Summary

Concept	Key Point
System Counter	Master source of time
Shared Counter	Common timeline for all cores
Monotonic	Never moves backwards
Fixed Frequency	Independent of CPU clock
Always-On	Runs through low-power states
Foundation	All timers derive time from it

Knowledge Check

Q1

What is the purpose of the System Counter?

Q2

Why did ARM choose a shared System Counter?

Q3

What does monotonic mean?

Q4

Why is the System Counter independent of CPU frequency?

Q5

Why is the System Counter usually located in an Always-On domain?

Q6

Why can two reads of the counter return different values?

Answer Key

Question	Answer
Q1	Provide a common source of time
Q2	Ensure all cores share the same timeline
Q3	Counter only moves forward
Q4	Stable timekeeping despite CPU frequency changes
Q5	Time must continue advancing during low-power states
Q6	The counter continues advancing between reads